BRIDGEWATER INTERNATIONAL COLLEGE

SINAMANGAL, KATHMANDU

A PROJECT SUBMITTED FOR

Final Assessment Report

COMPUTER SCIENCE

PREPARED BY

Bishwa Bastola

Samrin Dhakal

Gaurav Bhattrai

Aman Chand

MARCH 27, 2026

1. **Write a JavaScript function to multiply two numbers.**

Answer:

```
function multiply(a, b) {
  return a * b;
}

// Example usage:
const result = multiply(5, 4);
console.log(result); // Output: 20
```

```
OUTPUT
20
```

Explanation:

This function named `multiply` takes two parameters, `a` and `b`. It uses the multiplication operator (`*`) to multiply the two values together. The `return` statement passes the calculated product back to the caller. In the example usage, we call the function with arguments `5` and `4`, which evaluates to `20`.

2. **Write a program to read marks in Computer Science and display grade obtained using JavaScript.**

Given:

- marks >= 90 ---- grade 'A+'
- marks >= 80 ---- grade 'A'
- marks >= 70 ---- grade 'B+'
- marks >= 60 ---- grade 'B'
- marks >= 50 ---- grade 'C+'
- marks >= 40 ---- grade 'C'
- marks < 40 ---- grade 'F'

Answer:

```
function getGrade(marks) {
  if (marks >= 90) {
    return 'A+';
  } else if (marks >= 80) {
    return 'A';
  } else if (marks >= 70) {
    return 'B+';
  } else if (marks >= 60) {
    return 'B';
  } else if (marks >= 50) {
    return 'C+';
  } else if (marks >= 40) {
    return 'C';
  } else {
    return 'F';
  }
}

// Read marks (example input)
const marksInComputerScience = 78;

// Display the grade
const grade = getGrade(marksInComputerScience);
console.log("Marks:", marksInComputerScience);
console.log("Grade Obtained:", grade);
```

```
OUTPUT
Marks: 78
Grade Obtained: B+
```

Explanation:

The function `getGrade` utilizes a series of `if...else if` statements to evaluate the numerical `marks` against the defined grading conditions. Because the conditions check in descending order (highest marks to lowest), it correctly captures the corresponding grade as soon as a condition is met. For example, if `marks` is `78`, it skips the first two checks (since it's not ≥90 or ≥80) and evaluates to true at the `marks >= 70` condition, returning **'B+'**.

3. Program to find the factorial of a number using function

Answer:

```
#include<stdio.h>
#include<conio.h>
long int fact;
long int factorial(int n);

void main()
{
    int n;
    printf("Enter the number:");
    scanf("%d",&n);
    if(n<0)
        printf("Factorial of negative integer does not exist");
    else
    {
        fact=factorial(n);
        printf("%d! = %ld",n,fact);
    }
    getch();
}

long int factorial(int n)
{
    int i;
    fact=1;
    for(i=1;i<=n;i++)
        fact=fact*i;
    return(fact);
}
```

```
OUTPUT
Enter the number:5
5! = 120
```

Explanation:

This C program computes the factorial of a given integer using a separate user-defined function named `factorial()`. The `factorial()` function executes a `for` loop, continuously multiplying the numbers from `1` up to `n`. Notice that `fact` is declared as a global `long int` to prevent overflow for slightly larger factorials, and the main function also explicitly prevents invalid negative inputs.

4. Write a C program to generate Fibonacci series up to n terms using recursive function.

Answer:

```
/*Program to generate Fibonacci series up to n terms using recursive function*/
#include <stdio.h>

int fib(int n);

void main() {
    int n, i;
    printf("How many terms? ");
    scanf("%d", &n);
    printf("Fibonacci numbers up to %d terms:\n", n);
    for(i = 1; i <= n; i++)
        printf("%d\t", fib(i));
}

int fib(int n) {
    if(n == 1)
        return 0;
    else if(n == 2)
        return 1;
    else
        return (fib(n - 1) + fib(n - 2));
}
```

```
OUTPUT
How many terms? 6
Fibonacci numbers up to 6 terms:
0    1    1    2    3    5
```

Explanation:

The program prompts the user to enter the number of terms (`n`) they want to generate for the Fibonacci sequence.

`fib(int n)` Function: This recursive function calculates the nth value in the Fibonacci sequence. It establishes two base cases: if `n == 1`, it returns 0; if `n == 2`, it returns 1. For any other number, it recursively calls itself to return the sum of the two preceding terms using the formula `fib(n - 1) + fib(n - 2)`.

> **`main()` Function:** The `for` loop runs from `i = 1` up to `n`, calculating and printing the return value of `fib(i)` at each step to generate the sequential terms.

5. **Write a program which reads name, department and age from file named employee.dat and display them on monitor.**

Answer:

```
#include <stdio.h>
#include <stdlib.h>

struct employee {
    char name[30];
    char department[30];
    int age;
};

int main() {
    FILE *fptr;
    int i, n = 3; // Processing 3 records for example
    struct employee et[10], e[10];

    // Open file in "w+b" mode (write + read in binary)
    fptr = fopen("employee.dat", "w+b");
    if(fptr == NULL) {
        printf("File cannot be created\n");
        exit(0);
    }

    // Read from user and write to file
    for(i = 0; i < n; i++) {
        printf("Enter information of employee No. %d\n", i + 1);
        printf("Name: ");
        scanf("%s", et[i].name);
        printf("Department: ");
        scanf("%s", et[i].department);
        printf("Age: ");
        scanf("%d", &et[i].age);
    }

    printf("\nWriting information to the file...\n");
    fwrite(et, sizeof(struct employee), n, fptr);

    // Reset file pointer to the beginning before reading
    rewind(fptr);

    // Reading data from file
    printf("\nReading content from file:\n");
    fread(e, sizeof(struct employee), n, fptr);
```

```
    // Displaying data on monitor
    printf("\nName\t\tDepartment\tAge\n");
    printf("-----------------------------------\n");
    for(i = 0; i < n; i++) {
        printf("%-15s %-15s %d\n", e[i].name, e[i].department, e[i].age);
    }

    fclose(fptr);
    return 0;
}
```

```
OUTPUT
Enter information of employee No. 1
Name: Alice
Department: HR
Age: 29

Writing information to the file...
Reading content from file:

Name            Department      Age
-----------------------------------
Alice           HR              29
```

Explanation:

The program models an employee using `struct employee` containing `name`, `department`, and `age`.

It opens the file `employee.dat` using `fopen()` in **`w+b`** mode, securely allowing both writing and reading in binary format. After user inputs are aggregated in the `et` array struct, they are dumped concurrently into the file using `fwrite()`.

Crucially, `rewind(fptr)` resets the internal pointer to the beginning of the file so `fread()` can sequentially pull the data back into the secondary array `e`. Finally, the data is iterated through to neatly display the extracted records.

6. **Write a program to read roll, name and mark in computer of students from file "student.txt" and then display in tabular format.**

Answer:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fptr;
    char name[20];
    int roll;
    float mc;

    // Open file in read mode
    fptr = fopen("student.txt", "r");
    if(fptr == NULL) {
        printf("Error reading file...\n");
        exit(0);
    }

    printf("Roll\tName\tMark\n");
    printf("-----------------------\n");

    // Read formatted data until End Of File (EOF)
    while(fscanf(fptr, "%d %s %f", &roll, name, &mc) != EOF) {
        printf("%d\t%s\t%.2f\n", roll, name, mc);
    }

    fclose(fptr);
    return 0;
}
```

```
OUTPUT
Roll    Name      Mark
-----------------------
101     Aman      45.50
102     Gita      39.00
```

Explanation:

The program utilizes `fopen("student.txt", "r")` to open the text file exclusively in **read mode**.

> An unconditional error check is performed immediately: if `fptr` evaluates to `NULL`, the program gracefully prints an error and exits to prevent crashing.
>
> The `while` loop leverages `fscanf()` to repeatedly pluck formatted and structured text values from the file stream. It knows exactly how to capture an integer, a string, and a float concurrently. It reads variables natively in chunks until it safely encounters the natural `EOF` (End Of File) signal.
>
> Lastly, utilizing `\t` correctly snaps the output metrics to uniformly-spaced tabular columns on the visual monitor.